top КОЛЛЕДЖ

Автоматизация сборки Unix Makefiles

Операционные системы и среды

Дисклеймер

В данной презентации будут подряд идти задания, который вам необходимо выполнить на виртуальной машине, а также **теоретический материал**.

Удачи!



top КОЛЛЕДЖ

Автоматизация сборки

Как быстро собрать программу одной командой

Подготовка

top КОЛЛЕДЖ

- Далее будут идти слайды из презентации **Практика №6 «GASM – основы»**.
- Перед тем, как перейти к выполнению задания убедитесь, что у вас установлен компилятор ассемблера и другие необходимые утилиты. Если нет, вам необходимо установить пакет `binutils`.
- Также создайте файл с простым исходным кодом на GASM и расширением `.s`. В презентации в качестве имени файла везде используется `main.s`.



Автоматизация компиляции

top КОЛЛЕДЖ

- Вы могли заметить, что каждый раз перекомпилировать приложение с использованием двух команд может отнимать больше времени, чем хотелось бы.

```
as -o main.o main.s
ld -o main main.o
```

- *Возможно, некоторые из вас даже успели набросать скрипт на Bash, который эти команды поочередно выполняет.* ☺
- Мы сейчас сделаем именно это, но параллельно познакомимся с другой удобной утилитой для автоматизации сборки приложений в Unix-подобных ОС, которая называется ***Makefile***.

Makefile

top КОЛЛЕДЖ

- ***Makefile*** по сути представляет собой командный файл, который позволяет в простом виде указывать *цели сборки* (англ. *targets*) и зависимости между ними.
- Цель может быть или некоторым файлом (например, исходного кода), или абстрактной целью.
- Это может быть использовано не только для контроля последовательности компиляции исходного кода в исполняемый файл, но и для того, чтобы можно было увеличить скорость компиляции, так как можно определить, не изменился ли какой-то файл, чтобы не потребовалось его перекомпилировать лишний раз.

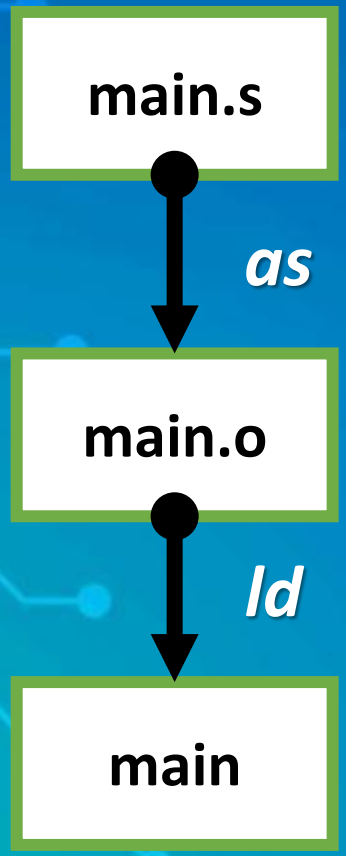
Makefile

top КОЛЛЕДЖ

- Для использования Makefile необходимо в каталоге проекта создать файл с названием `Makefile`.
- Makefile имеет следующий простой синтаксис:

```
{цель}: [зависимость…]
	{команда}
```

**** Серый пунктир означает табуляцию***



А это дерево зависимостей

Makefile

top КОЛЛЕДЖ

- Итак, наш Makefile для сборки приложения будет иметь следующий компактный вид.
- Директива .PHONY нужна для указания целей, которые являются абстрактными и не зависят от файлов. В нашем случае это цель `clean`, удаляющая ненужные объектные файлы.

```
main.o: main.s
	as -o main.o main.s

main: main.o
	ld -o main main.o

clean:
	rm -f *.o

build: main clean

.PHONY: clean build
```

Makefile

top КОЛЛЕДЖ

- Теперь мы можем компилировать наше приложение одной командой. Для использования сборки с помощью Makefile необходимо прописать команду

```
make {цель}
```

☼ Укажем абстрактную цель `build`. Тогда команда

```
make build
```

начнет выполнять цель `main` (сначала проверит зависимость `main.o`, та, в свою очередь, — изменение файла `main.s`, и при необходимости последовательно вызовет компилятор `as` и компоновщик `ld`), а затем выполнит цель `clean` (для удаления объектных файлов).